STQD6114 — Text Analytics Exam Reference Sheet

Question & Model Answer Reference: Reading Data, LDA, Text Clustering, Wordclouds, Sentiment Analysis

How to use this in the open-book exam: find the topic that matches the question, copy the R block, and only change the file path(s) marked `# <-- CHANGE THIS PATH`.

Setup — packages used across this sheet

```r
# Run once at the start of the exam
pkgs <- c("tm","wordcloud","wordcloud2","RColorBrewer","topicmodels","tidytext",
          "tidyr","dplyr","ggplot2","proxy","dbscan","cluster","colorspace",
          "ggrepel","ape","syuzhet","stringr","SnowballC")
# install.packages(pkgs)   # uncomment if a package is missing
invisible(lapply(pkgs, library, character.only = TRUE))
```

TOPIC 1 — Data Source Types: DirSource vs VectorSource vs DataframeSource (Concept)

Explain the three ways `tm` can build a corpus and when to use each.

Model Answer:

Source function	Input	When to use
`DirSource(path)`	A folder containing one `.txt` file per document	You have many separate article/review files sitting in a folder (e.g. `News_Articles/`)
`VectorSource(x)`	A character **vector**, one string per document	Your text already lives in R as a vector — e.g. one line per review from `readLines()`, or a column pulled out of a data frame/scraped data
`DataframeSource(df)`	A data frame with a column named `text` (and optionally `doc_id`)	Your text sits inside a CSV/data frame alongside metadata columns (author, date, rating) that you want kept as document metadata

```r
library(tm)

# 1. DirSource — folder of .txt files
corpus_dir <- VCorpus(DirSource("path/to/News_Articles"))

# 2. VectorSource — character vector already in R
my_vector  <- readLines("path/to/TeamHealthSentiment.txt")
corpus_vec <- VCorpus(VectorSource(my_vector))
```

```
# 3. DataframeSource — data frame with doc_id + text columns
my_df <- data.frame(doc_id = 1:3,
                    text   = c("first review", "second review", "third review"))
corpus_df <- VCorpus(DataframeSource(my_df))
```

Interpretation: All three converge on the same object type (`VCorpus`), so every cleaning/DTM/LDA/clustering step downstream is identical regardless of source — only the first line of the script changes depending on how the raw data arrives.

TOPIC 2 — Reading Data from a Directory (DirSource) for LDA/Clustering

Dataset: `Exam_Prep/News_Articles/` (folder of `.txt` news articles)

Read every `.txt` file in a folder into a single corpus, ready for cleaning.

Model Answer:

```
library(tm)

mytext   <- DirSource("path/to/News_Articles")   # <-- CHANGE THIS PATH
mycorpus <- VCorpus(mytext)

length(mycorpus)          # number of documents read in
mycorpus[[1]]$meta$id     # filename of the 1st document
```

Interpretation: `DirSource` treats every file in the folder as one document; `VCorpus` wraps them into a volatile corpus (held in memory) so `tm_map` transformations can be applied. This is the standard entry point for LDA and text-clustering questions where the data is "a folder of articles."

TOPIC 3 — Reading Data from a Vector (VectorSource) for Sentiment Analysis

Dataset: `TeamHealthSentiment.txt` (one free-text response per line)

Read a plain text file where every line is one document (e.g. one survey response) into a corpus.

Model Answer:

```
library(tm)

# Simplest case: one document per line
raw_lines <- readLines("path/to/TeamHealthSentiment.txt")   # <-- CHANGE THIS PATH
mytext    <- VectorSource(raw_lines)
mycorpus  <- VCorpus(mytext)

length(mycorpus)
```

```
as.character(mycorpus[[1]])

# --- Alternative: if the file/CSV is messy (rows split across many
#     columns), read with file.choose(), transpose, and re-glue each row ---
# data   <- read.table(file.choose(), fill = TRUE, header = FALSE)
# data   <- t(data)
# data_1 <- sapply(1:ncol(data), function(x) {
#             trimws(paste(data[, x], collapse = " "), which = "right")
#           })
# mycorpus <- VCorpus(VectorSource(data_1))
```

Interpretation: `VectorSource` is used whenever the raw text is already a character vector in R (one element = one document) rather than separate files on disk. `read.table` + transpose is only needed when a CSV/text file splits a single response across multiple columns because of stray delimiters.

TOPIC 4 — Reading Data from a Data Frame (DataframeSource) with Metadata

Dataset: A CSV/data frame of documents that also carries metadata (e.g. `doc_id`, `author`, `text`)

Build a corpus from a data frame while preserving non-text columns as document metadata.

Model Answer:

```
library(tm)

# DataframeSource requires a data frame with (at minimum) a column named "text";
# a "doc_id" column, if present, becomes the document ID
reviews_df <- read.csv("path/to/reviews.csv", stringsAsFactors = FALSE)  # <-- CHANGE THIS PATH
# reviews_df must have columns: doc_id, text, (any other metadata e.g. author, rating)

mycorpus <- VCorpus(DataframeSource(reviews_df[, c("doc_id", "text")]))

# metadata from the other columns can be attached manually if needed
meta(mycorpus[[1]], "author") <- reviews_df$author[1]
meta(mycorpus[[1]])
```

Interpretation: `DataframeSource` is the right choice when text arrives inside a spreadsheet/CSV alongside structured columns you want to keep tied to each document (e.g. reviewer name, star rating, date) instead of discarding them, unlike `VectorSource` which only keeps the raw text.

TOPIC 5 — Standard Text Cleaning Pipeline + Inspecting the Corpus

Dataset: Any corpus built from Topics 2–4 (e.g. `News_Articles`)

Apply the standard cleaning pipeline (numbers, punctuation, case, stopwords, custom stopwords, curly quotes/dashes), then check the corpus both **document-by-document** and **as a whole** for any remaining stray characters.

Model Answer:

```
library(tm)

toSpace <- content_transformer(function(x, pattern) { gsub(pattern, " ", x) })

docs <- tm_map(mycorpus, removeNumbers)
docs <- tm_map(docs, removePunctuation)
docs <- tm_map(docs, content_transformer(tolower))
docs <- tm_map(docs, removeWords, stopwords("english"))
docs <- tm_map(docs, removeWords, c("can","will","however","one","title","min",
    "read","including","also","new","may","mindich","said"))
docs <- tm_map(docs, toSpace, "–")
docs <- tm_map(docs, toSpace, "”")
docs <- tm_map(docs, toSpace, "“")
docs <- tm_map(docs, toSpace, "’")
docs <- tm_map(docs, toSpace, "‘")
docs <- tm_map(docs, toSpace, "-")

docs <- tm_map(docs, stripWhitespace)

# ---- 1) Inspect DOCUMENT BY DOCUMENT (spot-check a handful) ----
inspect(docs[[1]])
inspect(docs[[2]])
as.character(docs[[1]])          # content only, no metadata

for (i in 1:5) {
  print(as.character(docs[[i]]))
}

# ---- 2) Inspect the WHOLE CORPUS at once ----
all_text <- unname(sapply(docs, as.character))
head(all_text)                   # eyeball a few full documents
length(all_text)                 # confirm doc count unchanged after cleaning

# ---- 3) Scan the whole corpus for characters that still need removing ----
# any character that is NOT a lowercase letter, space, or digit
leftover_chars <- unique(unlist(regmatches(all_text,
                                           gregexpr("[^a-z0-9 ]", all_text))))
sort(leftover_chars)

# if leftover_chars shows things like "…" "—" "'" etc, add them to toSpace
# e.g. docs <- tm_map(docs, toSpace, "…")
```

Interpretation: `content_transformer` is required to wrap custom `gsub`-based functions so `tm_map` can apply them; base `tm` transformations (`removeNumbers`, `removePunctuation`, etc.) don't need this wrapper. Checking `inspect()`/`as.character()` on a few documents catches obvious issues, but scanning the **entire**

corpus for leftover non-alphanumeric characters (via `regmatches`/`gregexpr`) is what actually reveals hidden symbols (smart quotes, en/em dashes, ellipses) that the initial `toSpace` calls missed — these should be added back into the pipeline and the cleaning re-run.

TOPIC 6 — Document-Term Matrix & Frequency Analysis

Dataset: Cleaned corpus from Topic 5

Build a Document-Term Matrix, get overall term frequencies, and find frequent/associated terms.

Model Answer:

```
library(tm)

dtm <- DocumentTermMatrix(docs,
                          control = list(wordLengths = c(2, 20),   # keep 2-20
letter words
                                         bounds = list(global = c(1, 30))))
inspect(dtm)

freq <- colSums(as.matrix(dtm))
ord  <- order(freq, decreasing = TRUE)
freq[ord][1:20]                          # top 20 most frequent terms

df <- data.frame(TERM = names(freq), FREQ = freq)
df <- df[order(-df$FREQ), ]
head(df, 10)

findFreqTerms(dtm, lowfreq = 5)          # words appearing at least 5 times
findAssocs(dtm, "nasa", 0.3)             # words correlated with "nasa" at r >=
0.3
```

Interpretation: The DTM is the numeric backbone for every downstream step (LDA, clustering). `wordLengths`/`bounds` control noise by dropping single letters and extremely rare/overly common terms. `findAssocs` reveals words that tend to co-occur with a target term across documents, useful for validating that cleaning worked as intended.

TOPIC 7 — Basic Wordcloud from Term Frequencies

Dataset: `freq` table from Topic 6

Produce a simple wordcloud of the whole corpus's most frequent terms.

Model Answer:

```
library(wordcloud)
library(RColorBrewer)
```

```
set.seed(1234)
wordcloud(names(freq), freq,
          min.freq = 2,
          max.words = 100,
          random.order = FALSE,
          rot.per = 0.35,
          colors = brewer.pal(8, "Dark2"))

# quick alternative with wordcloud2 (interactive HTML widget)
# library(wordcloud2)
# wordcloud2(df, size = 0.5)
```

Interpretation: `random.order = FALSE` places the most frequent words in the centre, making the dominant themes visually obvious at a glance; `min.freq` filters out one-off words that would just clutter the plot.

TOPIC 8 — LDA Topic Modelling (k = 2 topics) on News Articles

Dataset: `News_Articles/` (DirSource, cleaned per Topic 5)

Fit a 2-topic LDA model, extract the per-topic-per-word probabilities (`beta`), and plot the top 15 terms per topic.

Model Answer:

```
library(topicmodels)
library(tidytext)
library(dplyr)
library(ggplot2)

dtm <- DocumentTermMatrix(docs)          # from the cleaned corpus, Topic 5

ap_lda <- LDA(dtm, k = 2, control = list(seed = 1234))   # 2-topic LDA model

ap_topics <- tidy(ap_lda, matrix = "beta")               # per-topic-per-word
probabilities

ap_top_terms <- ap_topics %>%
  group_by(topic) %>%
  top_n(15, beta) %>%
  ungroup() %>%
  arrange(topic, -beta)

ap_top_terms %>%
  mutate(term = reorder(term, beta)) %>%
  ggplot(aes(term, beta, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~topic, scales = "free") +
  coord_flip()
```

Interpretation: `beta` is the probability of a word being generated by a given topic. Facetting the bar chart by topic and using `scales = "free"` lets each topic show its own most-informative words on its own axis (e.g. one topic may surface "nasa", "artemis", "launch"; the other "government", "war", "president").

TOPIC 9 — Word Clouds per LDA Topic

Dataset: `ap_topics` (beta table) from Topic 8

Generate one wordcloud per topic so each topic's theme is visually clear.

Model Answer:

```
library(dplyr)
library(wordcloud)
library(RColorBrewer)

topic1_data <- ap_topics %>% filter(topic == 1) %>% top_n(100, beta)
set.seed(1234)
wordcloud(words = topic1_data$term, freq = topic1_data$beta,
          scale = c(3.5, 0.5), max.words = 100,
          random.order = FALSE, rot.per = 0.35,
          colors = brewer.pal(8, "Dark2"))

topic2_data <- ap_topics %>% filter(topic == 2) %>% top_n(100, beta)
set.seed(1234)
wordcloud(words = topic2_data$term, freq = topic2_data$beta,
          scale = c(3.5, 0.5), max.words = 100,
          random.order = FALSE, rot.per = 0.35,
          colors = brewer.pal(8, "Accent"))
```

Interpretation: Because `beta` is a probability (not a raw count), it can be fed directly into `wordcloud()`'s `freq` argument — larger `beta` values render larger words, letting each topic's wordcloud act as a quick "label" for that topic.

TOPIC 10 — LDA Beta Spread (Log-Ratio) Between Topics

Dataset: `ap_topics` from Topic 8

Find the terms with the greatest difference in probability between Topic 1 and Topic 2 using the log2 ratio of betas.

Model Answer:

```
library(dplyr)
library(tidyr)
library(ggplot2)

beta_spread <- ap_topics %>%
```

```
  mutate(topic = paste0("topic", topic)) %>%
  spread(topic, beta) %>%
  filter(topic1 > 0.003 | topic2 > 0.003) %>%
  mutate(log_ratio = log2(topic1 / topic2))

beta_spread %>%
  mutate(term = reorder(term, log_ratio)) %>%
  ggplot(aes(term, log_ratio)) +
  geom_col(show.legend = FALSE) +
  coord_flip()
```

Interpretation: A positive log-ratio means the term is far more associated with topic 1 than topic 2 (and vice-versa for negative). Filtering on `beta > 0.003` first keeps the chart to relatively common, meaningful words instead of being swamped by rare terms with extreme but meaningless ratios.

TOPIC 11 — LDA Document-Topic Probabilities (Gamma) & Document Word Check

Dataset: `ap_lda` model from Topic 8, plus original `dtm`

Extract the per-document-per-topic probabilities and identify the most common words in one specific document.

Model Answer:

```
library(tidytext)
library(dplyr)

tm_documents <- tidy(ap_lda, matrix = "gamma")   # per-document-per-topic
probabilities
print(n = 20, tm_documents)

# documents most strongly assigned to topic 2
tm_documents %>% filter(topic == 2) %>% arrange(desc(gamma))

# most common words in a specific document (by filename, since DirSource keeps the
filename)
tidy(dtm) %>%
  filter(document ==
"Science_Technology_NASA_Rolls_Out_Artemis_III_Moon_Rocket_Core_Stage.txt") %>%
  arrange(desc(count))
```

Interpretation: `gamma` gives each document's estimated proportion belonging to each topic (they sum to 1 per document); a document with `gamma ≈ 1` for one topic is almost purely about that topic, while a mixed document (e.g. 0.5/0.5) discusses both themes — this is the key advantage of LDA's "soft" topic assignment over hard clustering.

TOPIC 12 — Text Clustering Setup: TF-IDF Weighting & Cosine Distance

Dataset: `dtm` built from `News_Articles` (Topic 6)

Convert the DTM to TF-IDF weights, trim sparse terms, and compute a cosine distance matrix — the standard preprocessing before any text-clustering algorithm.

Model Answer:

```
library(proxy)   # for cosine distance

tdm <- dtm
tdm.tfidf <- weightTfIdf(tdm)                        # TF-IDF normalisation
tdm.tfidf <- removeSparseTerms(tdm.tfidf, 0.999)     # drop very sparse/rare terms

tfidf.matrix <- as.matrix(tdm.tfidf)
dist.matrix  <- dist(tfidf.matrix, method = "cosine")
```

Interpretation: TF-IDF down-weights words that appear in almost every document (uninformative, e.g. generic nouns) while up-weighting words that are distinctive to a smaller set of documents. Cosine distance is preferred over Euclidean for text because it measures the *angle* between term-frequency vectors (i.e. relative word-usage pattern), which is insensitive to document length.

TOPIC 13 — Elbow Method to Choose the Number of Clusters (K)

Dataset: `tfidf.matrix` from Topic 12

Run K-means for K = 1 to 20 and plot total within-cluster sum of squares to find the "elbow."

Model Answer:

```
cost_df <- data.frame()

for (i in 1:20) {
  set.seed(123)
  kmeans_test <- kmeans(x = tfidf.matrix, centers = i, nstart = 20, iter.max =
100)
  cost_df <- rbind(cost_df, cbind(i, kmeans_test$tot.withinss))
}
names(cost_df) <- c("cluster", "cost")

plot(cost_df$cluster, cost_df$cost, type = "b", pch = 19,
     xlab = "Number of Clusters (K)", ylab = "Total Within-Cluster Cost",
     main = "Elbow Plot for Optimal K Selection", col = "darkblue", lwd = 2)
axis(1, at = 1:20, labels = 1:20)
```

Interpretation: Total within-cluster sum of squares always decreases as K increases, so we don't pick the minimum — we pick the K where the curve visibly "bends" (diminishing returns), since adding more clusters

beyond that point barely improves the fit.

TOPIC 14 — K-Means Clustering & Visualisation

Dataset: `tfidf.matrix` / `dist.matrix` from Topic 12, K chosen from Topic 13 (e.g. K = 3)

Fit K-means with the chosen K and visualise the partition.

Model Answer:

```
library(cluster)

truth.K <- 3
set.seed(123)
clustering.kmeans <- kmeans(tfidf.matrix, centers = truth.K, nstart = 20)

table(clustering.kmeans$cluster)          # documents per cluster

clusplot(as.matrix(dist.matrix), clustering.kmeans$cluster,
         color = TRUE, shade = TRUE, labels = 0, lines = 0,
         main = "K-Means Cluster Partition Plot")

# 2D projection for a cleaner ggplot view
points <- cmdscale(dist.matrix, k = 2)
plot(points, col = clustering.kmeans$cluster, pch = 19,
     main = "K-Means Clustering (K=3)", xlab = "Dim 1", ylab = "Dim 2")
```

Interpretation: `cmdscale` (classical multidimensional scaling) projects the high-dimensional cosine-distance space down to 2D purely for visualisation — the cluster assignments themselves come from K-means run on the full TF-IDF matrix, not the 2D points.

TOPIC 15 — Hierarchical Clustering & Dendrogram

Dataset: `dist.matrix` from Topic 12

Perform Ward's-method hierarchical clustering and cut the tree into the same number of clusters as Topic 14.

Model Answer:

```
clustering.hierarchical <- hclust(dist.matrix, method = "ward.D2")

plot(clustering.hierarchical, main = "Cluster Dendrogram", xlab = "", sub = "")
rect.hclust(clustering.hierarchical, k = 3, border = "red")

h_assignments <- cutree(clustering.hierarchical, k = 3)
table(h_assignments)
```

Interpretation: Unlike K-means, hierarchical clustering doesn't need K decided up front — it builds a full nested tree, and `cutree`/`rect.hclust` just cut that tree at the height that yields the desired number of groups. Reading the dendrogram: lower merge height = more similar documents; the *height* is what matters, not the horizontal ordering of leaves.

TOPIC 16 — HDBSCAN Density-Based Clustering

Dataset: `dist.matrix` from Topic 12

Fit HDBSCAN, which does not require specifying K and can flag outliers as "noise."

Model Answer:

```
library(dbscan)

clustering.dbscan <- hdbscan(dist.matrix, minPts = 3)

table(clustering.dbscan$cluster)     # cluster 0 = unassigned noise

points <- cmdscale(dist.matrix, k = 2)
plot(points, col = clustering.dbscan$cluster + 1L, pch = 19,
     main = "HDBSCAN Density Clustering", xlab = "Dim 1", ylab = "Dim 2")
```

Interpretation: HDBSCAN looks for regions of higher density than their surroundings rather than assuming spherical, equally-sized clusters like K-means. Cluster label `0` denotes "noise" — documents that don't clearly belong to any dense region — which is useful for spotting genuinely unusual/outlier articles.

TOPIC 17 — Compare K-Means vs Hierarchical vs HDBSCAN Side-by-Side

Dataset: Results from Topics 14–16

Plot all three clustering results on the same 2D projection for direct comparison.

Model Answer:

```
library(colorspace)

points <- cmdscale(dist.matrix, k = 2)
main_colors <- c("coral2", "skyblue3", "goldenrod3")

par(mfrow = c(1, 3), mar = c(4, 4, 4, 1))

plot(points, main = 'K-Means Clustering', col =
main_colors[clustering.kmeans$cluster],
     pch = 19, cex = 1.2, xlab = 'Dim 1', ylab = 'Dim 2')

plot(points, main = 'Hierarchical Clustering', col = main_colors[h_assignments],
     pch = 19, cex = 1.2, xlab = 'Dim 1', ylab = '')
```

```
db_pal <- c("darkgray", "coral2", "skyblue3", "goldenrod3")
plot(points, main = 'HDBSCAN Clustering', col = db_pal[clustering.dbscan$cluster +
1L],
     pch = 19, cex = 1.2, xlab = 'Dim 1', ylab = '')

par(mfrow = c(1, 1))     # reset layout

table(clustering.kmeans$cluster)
table(h_assignments)
table(clustering.dbscan$cluster)
```

Interpretation: All three algorithms are run on the *same* distance matrix so the comparison is fair. K-means and hierarchical clustering will always assign every document to a cluster; HDBSCAN may leave some documents unassigned (noise), which is often the more honest answer when the corpus contains genuinely mixed-topic or off-topic articles.

TOPIC 18 — Sentiment Analysis: Corpus Setup & Lexicon-Based Scoring

Dataset: `TeamHealthSentiment.txt` (one team-health survey response per line — a separate dataset from the News_Articles clustering/LDA questions)

Read the survey responses, clean them, and score sentiment with three different lexicons (syuzhet, bing, AFINN).

Model Answer:

```
library(tm)
library(syuzhet)

# --- Read: VectorSource, one response per line ---
raw_lines <- readLines("path/to/TeamHealthSentiment.txt")   # <-- CHANGE THIS PATH
mycorpus  <- VCorpus(VectorSource(raw_lines))

# --- Clean ---
toSpace <- content_transformer(function(x, pattern) { gsub(pattern, " ", x) })
docs <- tm_map(mycorpus, toSpace, "-")
docs <- tm_map(docs, toSpace, "'")
docs <- tm_map(docs, removePunctuation)
docs <- tm_map(docs, content_transformer(tolower))
docs <- tm_map(docs, removeWords, stopwords("english"))
docs <- tm_map(docs, removeNumbers)
docs <- tm_map(docs, stripWhitespace)

# inspect a few + the whole corpus (same practice as Topic 5)
as.character(docs[[1]])
unname(sapply(docs, as.character))[1:3]

corpus_text_vector <- sapply(docs, as.character)
```

```
# --- Score with three lexicons ---
syuzhet_vector <- get_sentiment(corpus_text_vector, method = "syuzhet")
bing_vector    <- get_sentiment(corpus_text_vector, method = "bing")
afinn_vector   <- get_sentiment(corpus_text_vector, method = "afinn")

summary(syuzhet_vector)
summary(bing_vector)
summary(afinn_vector)

rbind(Syuzhet = sign(head(syuzhet_vector)),
      Bing    = sign(head(bing_vector)),
      AFINN   = sign(head(afinn_vector)))
```

Interpretation: Each lexicon scores sentiment differently (syuzhet uses its own dictionary, bing is binary positive/negative word counts, AFINN uses a -5 to +5 word scale), so comparing their *sign* (positive/negative direction) across responses checks whether the overall sentiment conclusion is robust regardless of which lexicon is used, rather than relying on a single method.

TOPIC 19 — Sentiment Analysis: NRC Emotion Classification & Plot

Dataset: `corpus_text_vector` from Topic 18

Classify each response into 8 core emotions (NRC lexicon) and plot the overall emotion distribution.

Model Answer:

```
library(syuzhet)
library(ggplot2)

emotion_df <- get_nrc_sentiment(corpus_text_vector)
head(emotion_df, 10)

td     <- data.frame(t(emotion_df))
td_new <- data.frame(count = rowSums(td))
td_new <- cbind(sentiment = rownames(td_new), td_new)
rownames(td_new) <- NULL

emotions_only <- td_new[1:8, ]     # anger through trust (drop pos/neg totals)

ggplot(emotions_only, aes(x = reorder(sentiment, -count), y = count, fill =
sentiment)) +
  geom_bar(stat = "identity") +
  theme_minimal() +
  labs(title = "Team Health Survey — Emotion Distribution",
       x = "Emotion", y = "Word Association Count") +
  theme(legend.position = "none")
```

Interpretation: `get_nrc_sentiment` returns word counts across 10 columns (8 emotions + positive/negative); transposing and summing gives total word associations per emotion across the whole

corpus. A team-health dataset dominated by "trust" and "joy" over "anger"/"fear" indicates broadly positive team sentiment.

TOPIC 20 — Sentiment Analysis: Positive/Negative Words & Word Clouds

Dataset: `docs` (cleaned corpus) from Topic 18

Extract the most common positive and negative words (Bing lexicon) and render them as separate word clouds.

Model Answer:

```
library(dplyr)
library(tidytext)
library(wordcloud)
library(RColorBrewer)

text_df <- data.frame(text = corpus_text_vector, stringsAsFactors = FALSE)

word_tokens <- text_df %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words, by = "word")

bing_word_counts <- word_tokens %>%
  inner_join(get_sentiments("bing"), by = "word", relationship = "many-to-many")
%>%
  count(word, sentiment, sort = TRUE)

top_words <- bing_word_counts %>%
  group_by(sentiment) %>%
  slice_max(n, n = 20) %>%
  ungroup()

ggplot(top_words, aes(x = reorder(word, n), y = n, fill = sentiment)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~sentiment, scales = "free_y") +
  coord_flip() +
  labs(title = "Most Common Positive/Negative Terms", x = NULL, y = "Frequency")

# --- Word clouds ---
negative_words <- bing_word_counts %>% filter(sentiment == "negative")
positive_words <- bing_word_counts %>% filter(sentiment == "positive")

set.seed(123)
wordcloud(words = negative_words$word, freq = negative_words$n,
          min.freq = 1, max.words = 40, random.order = FALSE,
          colors = brewer.pal(8, "Reds")[4:8])
title(main = "Negative Themes", line = -1)

set.seed(123)
wordcloud(words = positive_words$word, freq = positive_words$n,
```

```
            min.freq = 1, max.words = 40, random.order = FALSE,
            colors = brewer.pal(8, "Greens")[4:8])
  title(main = "Positive Themes", line = -1)
```

Interpretation: `unnest_tokens` breaks each response into one-word-per-row (tidytext format), which is what allows a simple `inner_join` against the Bing sentiment dictionary. Splitting the resulting word clouds into positive vs negative surfaces the actual *drivers* behind the sentiment scores from Topic 18 (e.g. "trust", "fun", "collaborative" vs "frustrat-", "stress-", "uncertain-"), which is far more actionable than the raw score alone.

Quick Exam Checklist

- **Data sources:** `DirSource(path)` → folder of files; `VectorSource(vector)` → text already in R; `DataframeSource(df)` → data frame with a `text` column (+ optional `doc_id`/metadata).
- **Cleaning order that matters:** numbers → punctuation → lowercase → stopwords → custom stopwords → curly quotes/dashes (`toSpace`) → `stripWhitespace`.
- Always wrap custom regex cleaners in `content_transformer()` before passing to `tm_map`.
- Check cleaning both ways: a few docs via `inspect()`/`as.character()`, and the **whole corpus** via `unname(sapply(docs, as.character))` + a regex scan for leftover symbols.
- **LDA** needs a `DocumentTermMatrix`; extract `beta` (word-topic) and `gamma` (doc-topic) via `tidy(lda_model, matrix = "beta"/"gamma")`.
- **Text clustering** needs TF-IDF weighting + cosine `dist()` first; K-means needs K chosen (elbow method), hierarchical doesn't, HDBSCAN doesn't either (and can mark noise as cluster `0`).
- **Wordclouds** can be built from raw term frequency, LDA `beta`, or Bing positive/negative counts — same `wordcloud()` call each time, just a different `freq` source.
- **Sentiment analysis** is typically a *separate* dataset (survey/reviews) from the LDA/clustering news-articles data, but reuses the same `tm` cleaning pipeline before scoring with `syuzhet`/`bing`/`afinn`/NRC.

End of Reference — 20 Topics covering Data Sources, Cleaning, LDA, Text Clustering, Wordclouds, and Sentiment Analysis